Axia: Hierarchical Reasoning for Tiny/Small LLMs – Architecture and Methodology Guide

- Self-consistency combined with chain-of-thought (CoT) reasoning significantly improves small model performance on complex tasks, often by 10-20% on benchmarks like GSM8K.
- Tree-of-Thoughts (ToT) and Graph-of-Thoughts (GoT) provide structured exploration of multiple reasoning paths, improving robustness over single CoT chains.
- Retrieval-augmented generation (RAG) with local vector stores (e.g., Chroma, Qdrant) enables small models to incorporate external knowledge without retraining, critical for context-aware reasoning.
- Structured output enforcement via JSON Schema, Pydantic validation, and repair loops ensures reliable, auditable intermediate artifacts and final outputs.
- Comprehensive interaction data curation, including schema validation, critique loops, and replayable traces, is essential for future supervised fine-tuning and preference learning.

Introduction

Axia aims to enhance the performance of tiny and small language models (e.g., Ollama-compatible models) through deterministic hierarchical reasoning without modifying model weights during inference. By treating the model as a narrow semantic operator within a controlled pipeline, Axia will improve output quality via task decomposition, structured intermediate artifacts, schema validation, retrieval/memory integration, critique/repair loops, scoring gates, bounded retries, replayable traces, and final synthesis from auditable work. Additionally, Axia must include an explicit user-enabled mode for passively collecting high-quality interaction data to export later for supervised fine-tuning, preference tuning, evaluation, or retrieval.

This report synthesizes recent empirical research, benchmarks, and engineering best practices to guide Axia's architecture and implementation. It focuses on methods implementable outside model weights during inference, separating reasoning, retrieval, prompting, verification, and data curation layers to provide a rigorous, source-grounded technical foundation.

Tiny/Small Model Reasoning Behavior

Small language models (typically <10B parameters) exhibit distinct reasoning challenges compared to larger models. Empirical studies show that while chain-of-thought (CoT) prompting improves reasoning in large models, it often degrades performance in smaller

models due to incoherent or illogical reasoning chains [1]. Self-consistency, which samples multiple reasoning paths and selects the most consistent answer, has been proven to substantially improve CoT performance across arithmetic and commonsense reasoning benchmarks (e.g., GSM8K +17.9%, SVAMP +11.0%) [2 3 4 5 6]. This suggests that small models benefit from structured reasoning frameworks that guide and validate intermediate steps rather than relying solely on hidden CoT.

Small models also suffer from context-window weaknesses such as “lost in the middle” effects and position sensitivity, which impair their ability to utilize long contexts effectively [1]. Hallucination patterns and quantization effects further reduce reliability. Local inference constraints (latency, RAM/VRAM, context length, KV cache cost, batching limits) necessitate efficient orchestration and memory management to maximize performance.

Reasoning Enhancement Techniques

Chain-of-Thought (CoT) and Self-Consistency

CoT prompting elicits step-by-step reasoning by instructing the model to “think step by step.” While effective for large models, small models often struggle with CoT due to incoherent reasoning chains. Self-consistency improves CoT by generating multiple reasoning paths and selecting the most consistent answer, significantly boosting accuracy without additional supervision or fine-tuning [2 3 4 5 6]. This technique is highly applicable to Axia’s goal of improving small model reasoning reliability.

Tree-of-Thoughts (ToT) and Graph-of-Thoughts (GoT)

ToT explores multiple reasoning paths via tree-structured search (BFS/DFS) to systematically explore diverse sub-problems, improving robustness over linear CoT chains [7 8 9]. GoT extends this by integrating graph structures for complex problem-solving, enabling better handling of multi-step and multi-branch reasoning tasks. Both methods enhance small model performance by providing structured exploration and aggregation of reasoning paths.

ReAct and Multi-Agent Debate

ReAct interleaves reasoning with actions (e.g., tool calls) and reflection, enabling models to interact with external environments [10]. While variable in effectiveness, it can enhance reasoning by incorporating real-time feedback. Multi-agent debate and critique loops (e.g., answer-then-critique-then-repair) improve output quality by iteratively refining answers through structured critique and repair cycles [11].

Decomposition and Planner-Executor Architectures

Task decomposition breaks complex problems into smaller, atomic subtasks, enabling small models to handle complexity incrementally [12 13 14]. Planner-executor architectures separate

planning from execution, allowing specialized models or components to handle each phase, improving reliability and auditability.

Memory and Context-Awareness Tools

Retrieval-Augmented Generation (RAG)

RAG integrates external knowledge retrieval at inference time, enabling small models to access up-to-date or domain-specific information without retraining [15] [16]. Local vector stores (e.g., Chroma, Qdrant, LanceDB) provide efficient, privacy-sensitive retrieval compatible with small models. RAG systems must balance precision, recall, and efficiency, with benchmarks like RAGAS evaluating retrieval quality and evidence grounding.

Graph Memory and Long-Term Memory Frameworks

Graph memory systems (e.g., MemGPT) manage memory hierarchically across RAM and disk storage, enabling flexible, context-sensitive retrieval and long-term memory management [17] [18]. The Stability- and Safety-Governed Memory (SSGM) framework proposes protocols for ensuring memory correctness and safety over time, critical for small models with limited internal state [19].

Local Embeddings and Privacy

Local embedding models (e.g., Ollama) allow running the entire embedding pipeline on-device, reducing API costs and data egress [20]. This is essential for privacy-sensitive deployments and aligns with Axia's local-first ethos.

Structured Output and Validation

JSON Mode and Schema Validation

JSON mode and schema validation enforce that model outputs strictly adhere to predefined structures, critical for reliable parsing and downstream integration [21] [22] [23]. JSON Schema defines validation rules, enabling deterministic output contracts that reduce hallucinations and format violations.

Repair Loops and Retry Mechanisms

Automatic repair loops (e.g., Instructor's retry logic) enable models to self-correct outputs that fail schema validation, improving reliability without developer intervention [24] [25] [26]. Streaming support with partial JSON validation allows progressive content delivery while maintaining structural integrity.

Tools Comparison

Tool	Schema Support	Retry Logic	Streaming	Provider Agnostic	Latency Overhead	Use Case Fit for Axia
Instructor	Yes	Yes	Yes	Yes	Low	High (recommended)
Outlines	Yes	Yes	Yes	Yes	Medium	Medium
Guidance	Yes	Yes	Yes	Yes	Low	High (grammar-based)
Jsonformer	Yes	No	No	No	High	Low
LMQL	Yes	No	No	No	Medium	Medium

Instructor and Guidance are recommended for Axia due to their robust schema support, retry mechanisms, streaming, and provider agnosticism [27] [28] [24] [25] [26].

Live Interaction Data Curation

Data Storage and Formats

Axia should store interactions in JSON Lines (.jsonl) format, supporting instruction-response pairs with optional fields for input, system, and history [29] [30]. This format is compatible with LLaMA-Factory and other fine-tuning frameworks.

Data Quality and Curation Pipeline

A comprehensive data curation pipeline should include:

- **Data Collection:** Raw user requests, normalized requests, inferred intent, constraints, task constitution, work graph, intermediate artifacts, retrieved context, tool calls/results, model profile/decoding parameters, prompts/prompt hashes, model outputs, validation errors, repair attempts, critique outputs, scorecards, accepted/rejected drafts, user edits/corrections, explicit ratings, implicit signals (ethical/locally defined), provenance/timestamps, privacy/redaction status, export eligibility, dataset split assignment [31] [29] [32].
- **Quality Assessment:** Rule-based filters, LLM quality judges, human reviews, and automated pipelines (e.g., CLEAR) to filter or correct low-quality data [33] [34].
- **Data Augmentation:** Tools like Nuklai can automate labeling, bias detection, synthetic data generation, and collaborative annotation to enrich datasets [35].

Dataset Format Conversions

Axia should support conversion to instruction/input/output JSONL, chat-format JSONL, Alpaca-style, ShareGPT-style, OpenAI fine-tuning chat format, preference pairs (DPO/ORPO/KTO), critique-revision pairs, rejected-vs-accepted pairs, retrieval triples, verifier examples, rubric-scored examples, tool-use traces, synthetic benchmarks from real tasks [29] [30] [36] [32].

Benchmarks and Evaluation

Relevant Benchmarks

- **GSM8K:** 1,319 grade school math word problems requiring multi-step reasoning [37] [38].
- **MMLU:** 57 subjects, multiple-choice questions evaluating broad knowledge [39] [40].
- **MMLU-Redux:** Manually re-annotated version of MMLU for improved quality [41].
- **GSM8K-Verification:** Methodological template for trustworthy evaluation and deployment of reasoning models [42].

Axia-Specific Evaluation Suite

Proposed metrics include:

- Final answer accuracy
- Schema validity rate
- Retry success rate
- Hallucination rate
- Contradiction rate
- Evidence grounding
- Trace replayability
- Cost/latency
- Memory retrieval quality
- Context-position sensitivity
- Small-model uplift vs. single-shot baseline
- Dataset quality
- Export completeness
- Training-example acceptance rate
- Preference-pair quality
- Correction usefulness

This suite will enable Axia to quantify improvements and ensure reliability across its hierarchical reasoning pipeline [41] [42] [11].

Architecture Recommendations for Axia MVP

Axia's architecture should be:

- **Local-first and provider-agnostic:** Use local models (e.g., Ollama) to minimize costs and ensure privacy, with a unified interface abstracting provider complexities [43] [44].
- **Modular and graph-based:** Leverage graph-based orchestration (e.g., LangGraph) for flexible workflows supporting conditionals, loops, branching, and error handling [45] [46] [47].
- **Memory system:** Implement a robust memory system managing primary context (RAM) and external context (disk storage) with archival and paging mechanisms [48].
- **Controller loop:** Orchestrate model calls, task decomposition, memory retrieval, schema validation, critique/repair loops, and final synthesis.
- **Interaction data capture:** Passively collect high-quality interaction data structured for future fine-tuning, preference tuning, and evaluation.
- **Deterministic fake provider:** For offline testing and validation.

This architecture ensures Axia is scalable, auditable, and adaptable to evolving model capabilities and user needs.

Comparison Matrix of Key Techniques/Tools

Technique/ Tool	Reasoning Uplift	Reliability Improvement	Small-Model Suitability	Implementation Complexity	Runtime Cost	Local-First Compatibility	Auditability	MVP Priority	Evide Stren
Chain-of-Thought (CoT)	High	High	Medium	Low	Low	Yes	Medium	High	High
Self-Consistency	High	High	High	Medium	Medium	Yes	High	High	High
Tree of Thoughts (ToT)	Medium	Medium	High	Medium	Medium	Yes	High	Medium	Mediu
ReAct	Variable	Variable	Medium	High	High	Yes	Medium	Medium	Mediu
Graph of Thoughts (GoT)	High	High	High	High	High	Yes	High	High	High
SE-GBS	High	High	High	High	High	Yes	High	High	High
REPS	High	High	High	Medium	Medium	Yes	High	High	High
CRP-RAG	High	High	High	Medium	Medium	Yes	High	High	High
SelfRewardRAG	High	High	High	Medium	Medium	Yes	High	High	High

Technique/ Tool	Reasoning Uplift	Reliability Improvement	Small-Model Suitability	Implementation Complexity	Runtime Cost	Local-First Compatibility	Auditability	MVP Priority	Evide Stren
TIR	High	High	Medium	Medium	Medium	Yes	High	Medium	Mediu
rStar-Math	High	High	High	High	High	Yes	High	High	High
Logic-RL	High	High	High	High	High	Yes	High	High	High

Final Deliverables Structure

1. **Executive Summary:** Key takeaways for Axia.
2. **Best Findings for Axia:** Top techniques, tools, and patterns.
3. **What Works for Tiny/Small Models:** Evidence-backed methods.
4. **What Likely Fails or Overpromises:** Techniques with weak/negative evidence for small models.
5. **Reasoning Technique Survey:** Detailed breakdown with citations.
6. **Memory/Context Technique Survey:** Tools and methods with assessments.
7. **Tool/Framework Survey:** Local-first compatibility, maturity, etc.
8. **Structured-Output and Validation Survey:** Methods and tools.
9. **Live Interaction Data Curation Survey:** Schemas, formats, and practices.
10. **Training/Fine-Tuning Dataset Schema Recommendations:** Axia's data model and export designs.
11. **Benchmark and Evaluation Plan:** Axia-specific suite and metrics.
12. **Recommended Axia MVP Architecture:** Modular, auditable, local-first design.
13. **Recommended Implementation Roadmap:** Phased priorities.
14. **Risks and Open Questions:** Gaps, uncertainties, failure modes.
15. **Annotated Bibliography:** Key sources with summaries.
16. **Source Links and Primary References:** Direct links to papers, docs, repos.

Conclusion

Axia's hierarchical reasoning system must integrate self-consistency-enhanced chain-of-thought, tree/graph-of-thoughts, retrieval-augmented generation with local vector stores, structured output with schema validation and repair loops, and comprehensive interaction data curation. The architecture should be local-first, provider-agnostic, and modular, enabling robust reasoning, memory management, and future fine-tuning data export. By focusing on techniques proven effective for small models and avoiding hidden CoT or immediate training during inference, Axia can significantly improve the performance, reliability, and auditability of tiny and small language models in production environments.

This report provides a rigorous, source-grounded technical foundation to guide Axia's planning, architecture, and implementation roadmap.

[1] Chain of Thought in Large Language Models: Elicited Reasoning or Constrained Imitation? | by Greg Robison | Medium
[2] [2203.11171] Self-Consistency Improves Chain of Thought Reasoning in Language Models
[3] Self-Consistency Improves Chain of Thought Reasoning in Language Models | OpenReview
[4] [PDF] Self-Consistency Improves Chain of Thought Reasoning in Language Models | Semantic Scholar
[5] Self-Consistency Improves Chain of Thought Reasoning in ...
[6] SELF-CONSISTENCY IMPROVES CHAIN OF THOUGHT ...
[7] Recursive Decomposition of Logical Thoughts: Framework for
[8] Demystifying Chains, Trees, and Graphs of Thoughts
[9] Chain of Draft: Thinking Faster by Writing Less
[10] A comparative evaluation of chain-of-thought-based prompt engineering techniques for medical question answering - ScienceDirect
[11] SLM-MUX: Orchestrating Small Language Models for Reasoning
[12] How to Create Task Decomposition
[13] An Approach for Systematic Decomposition of Complex LLM Tasks
[14] Advancing Agentic Systems: Dynamic Task Decomposition, Tool Integration and Evaluation using Novel Metrics and Dataset
[15] Long-Term Memory for LLMs: 2023 – 2025
[16] Memory in Large Language Models: Mechanisms, Evaluation and Evolution
[17] Memory for Autonomous LLM Agents: Mechanisms, Evaluation, and Emerging Frontiers
[18] Human-Like Remembering and Forgetting in LLM Agents: An ACT-R-Inspired Memory Architecture | Proceedings of the 13th International Conference on Human-Agent Interaction
[19] Governing Evolving Memory in LLM Agents: Risks, Mechanisms, and the Stability and Safety Governed Memory (SSGM) Framework
[20] Why Local LLMs Matter in 2025. Large language models running entirely... | by Daniel Tse | Medium
[21] The guide to structured outputs and function calling with LLMs
[22] Structured Output Generation in LLMs: JSON Schema and Grammar-Based Decoding | by Emre Karatas | Medium
[23] Structured Output Enforcement: LLM JSON & Format Control | Inference Systems
[24] Structured outputs | LLM Inference Handbook
[25] Generate structured output from LLMs with Dottxt Outlines in AWS | Artificial Intelligence
[26] Reliable JSON from Any LLM: Pydantic + Zod (2026) | TECHSY
[27] LLM Structured Outputs: Schema Validation for Real Pipelines (2026)
[28] Beyond JSON Mode: Getting Reliable Structured Outputs from LLMs in Production - TianPan.co
[29] High-Quality Dataset Curation for LLM Finetuning: Expert Guide | Artificial Intelligence in Plain English
[30] Dataset preparation for LLM post-training | Anyscale Docs
[31] How to Create Custom Instruction Datasets for LLM Fine-tuning
[32] LLM Fine-Tuning Data Pipeline: Collection, Annotation & Synthetic Data [2026] | Meta Intelligence
[33] Automated Data Curation for Robust Language Model Fine-Tuning
[34] [2403.12776] Automated Data Curation for Robust Language Model Fine-Tuning

[35] r/ArtificialInteligence on Reddit: LLM Fine-tuning Best Practices for Training Data Curation (discovered FT'ing thousands of models)
[36] Curating Custom Datasets for LLM Parameter-Efficient Fine-Tuning with NVIDIA NeMo Curator | NVIDIA Technical Blog
[37] GPT-5.1 Benchmarks: MMLU, GSM8K, and Real-World Tests - Skywork ai
[38] GSM8K | DeepEval - The LLM Evaluation Framework
[39] LLM Benchmarks Compared: MMLU, HumanEval, GSM8K and More (2026)
[40] 40 Large Language Model Benchmarks and The Future of Model Evaluation
[41] AI Benchmarks 2026: Compare 300+ LLM Benchmarks & Tests
[42] GSM8k Verification Research
[43] LLMOps blueprint for open-source large language models
[44] Implementing an LLM Agnostic Architecture | Entrio
[45] LLM Orchestration in 2026: Top 22 frameworks and gateways
[46] LLM Orchestration Frameworks: Architecture, Components, and Comparisons | by Pandiyaraj Selvaraj | Medium
[47] 9 Best LLM Orchestration Frameworks for Agents and RAG - ZenML Blog
[48] Design Patterns for Long-Term Memory in LLM-Powered Architectures